



TD 07 - Gitlab CI/CD

GitLab CI/CD (Continuous Integration / Continuous Deployment) est un outil puissant qui permet d'automatiser le processus de développement logiciel, du test au déploiement. Il repose sur des **pipelines**, composés de **jobs** s'exécutant dans des **stages**.

Dans cette série d'exercices, vous apprendrez à configurer un pipeline GitLab CI/CD, à automatiser les tests et à déployer des applications de manière efficace.

Objectifs

À l'issue de ce TD, vous serez capable déployer une application sur Alwaysdata en utilisant un pipeline GitLab CI/CD.

PRÉ-REQUIS

1. Connaissance de base en Java, Python, Docker et des commandes shell.
2. Un environnement de travail prêt avec Java (JDK 17 minimum), Python 3.8+, Docker Engine et un IDE.
3. Notions de base sur Git, GitLab et SonarQube.
4. Bases sur les fichiers YAML

Installation

Pour utiliser GitLab CI/CD, vous devrez :

1. Lancer un conteneur exécutant les instructions transmises par GitLab, appelé GitLab Runner.
2. Générer un token permettant à ce GitLab Runner de communiquer avec le serveur GitLab.
3. Enregistrer ce GitLab Runner, s'exécutant sur votre machine hôte, auprès du serveur GitLab.



DÉFINITION DE PIPELINE ET DE JOBS DANS GITLAB CI/CD

Un **pipeline** est un processus automatisé qui exécute une série de **jobs** définis dans un fichier `.gitlab-ci.yml`.

Un **job** est une unité d'exécution définie dans le fichier `.gitlab-ci.yml`. Il correspond à une tâche spécifique à exécuter dans le pipeline, comme :

- Exécuter des tests
- Construire un projet
- Déployer une application

Installation du runner

Un **GitLab Runner** est un programme qui exécute les tâches définies dans un pipeline GitLab CI/CD. Il prend en charge l'exécution des **jobs** spécifiés dans un fichier `.gitlab-ci.yml`.

Les runners peuvent être partagés, c'est à dire gérés par le serveur GitLab ou spécifiques et installés sur une machine privée. Ils s'exécutent dans différents environnements (Docker, shell, ...) et permettent d'automatiser le processus de développement.

Afin d'éviter d'installer les dépendances sur votre machine hôte et de garantir un environnement identique à chaque exécution, vous allez installer un runner sur votre machine dans un environnement Docker en utilisant l'image `gitlab-runner`. La procédure d'installation décrite ci-dessous s'inspire de [la documentation sur gitlab-runner](#). N'hésitez pas à la consulter si vous souhaitez découvrir plus de possibilités d'installation.

Pour faciliter la communication entre les services Docker et GitLab Runner, vous allez utiliser un **socket** unix.

SOCKET

Un socket est un point de communication utilisé pour échanger des données entre des processus. Il permet la communication entre applications, soit localement sur une même machine, soit à distance via un réseau.

En particulier `/var/run/docker.sock` est un socket UNIX utilisé pour communiquer avec le daemon Docker (*dockerd*) sans passer par une connexion réseau. Il permet aux processus locaux, comme un GitLab Runner ou un autre conteneur, de contrôler Docker sur la machine hôte.

Nous avons besoin d'un dossier qui servira de volume pour conserver la **configuration persistante du GitLab Runner**. Ce sera le dossier

`/chemin_absolu_vers_la_configuration/gitlab-runner/config`.

- Sur Linux, le dossier `/srv` existe par défaut sur votre machine hôte et est destiné à héberger des données de services (serveurs web, applications...). C'est lui qui sera le `chemin_absolu_vers_la_configuration`
- Sur certaines distributions minimalistes ou personnalisées, il peut être absent, et il faut alors le créer manuellement.
- Sur Windows créez un dossier `gitlab-runner/config` à l'emplacement de votre choix.

Exécutez les commandes ci-dessous pour télécharger puis démarrer le runner :

```
docker pull gitlab/gitlab-runner:latest
```

```
docker run -d --name gitlab-runner --restart always \
-v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-
runner \
-v //var/run/docker.sock:/var/run/docker.sock \
gitlab/gitlab-runner:latest
```

! GITBASH

Pour rappel, les commandes données fonctionnent sur Linux ainsi qu'avec GitBash sur Windows.

- Le `//` permet d'indiquer à GitBash que le chemin doit être compris comme un chemin absolu. (avec une simple `/` il est compris comme un chemin relatif par rapport au dossier d'installation de GitBash.) Si vous êtes sur Linux, vous pouvez mettre une simple `/` mais le `//` fonctionne aussi.
- Le `\` en fin de ligne indique que la commande n'est pas terminée; elle continue à la ligne suivante.
- Si vous travaillez sur une machine Windows de l'école et que vous avez créé le dossier `gitlab-runner` dans votre répertoire `Documents`, la commande suivante s'applique (en supposant que votre nom d'utilisateur est `g12345`) :

```
docker run -d --name gitlab-runner --restart always \
-v //c/Users/g12345/Documents/gitlab-runner/config:/etc/gitlab-
runner \
-v //var/run/docker.sock:/var/run/docker.sock \
gitlab/gitlab-runner:latest
```

- Si vous utiliser une CommandTool ou PowerShell sur Windows, il vous faudra adapter la commande.

Création du token pour un projet

Un GitLab Runner doit être enregistré auprès d'un projet GitLab pour exécuter ses pipelines. Pour cela, il a besoin d'un token d'enregistrement, qui sert à authentifier le runner et à l'associer au projet. Afin de créer ce token :

- Créez un projet [sur GitLab](#).
- Dans le menu du projet *Settings* > *CI/CD*, ouvrez la section *Runners*.
- Créez le runner via le bouton *New project runner*.
- Cochez la case *Run untagged jobs*.
- Copiez le runner authentication token avant qu'il ne disparaisse.
- **Exportez** la valeur du token dans la variable **RUNNER_TOKEN**.



POURQUOI COCHEZ RUN UNTAGGED JOBS ?

Les runners GitLab peuvent être associés à des tags pour mieux organiser l'exécution des jobs. Un job sans tags sera exécuté **uniquement** par un runner qui n'exige pas de tags.

Enregistrement d'un runner

Pour enregistrer le runner de votre machine hôte, il faut exécuter la commande `register` du conteneur `gitlab-runner` :

```
docker run --rm \
  -v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-
runner \
  gitlab/gitlab-runner register \
    --non-interactive \
    --url "https://git.esi-bru.be" \
    --token "$RUNNER_TOKEN" \
    --executor "docker" \
    --docker-image alpine:latest \
    --description "docker-runner"
```

EXERCICE A : INTERPRÉTER LA COMMANDE DOCKER RUN

Associez les descriptions ci-dessous aux différents paramètres de la commande précédente (`--non-interactive`, `--url "https://git.esi-bru.be"`, ...) :

- Fournit une description pour le runner, qui permet de l'identifier dans l'interface GitLab.
- Spécifie l'URL de votre instance GitLab.
- Utilise le token d'enregistrement contenu dans la variable d'environnement `$RUNNER_TOKEN`.
- Indique que l'enregistrement du runner doit se faire sans interaction utilisateur, ce qui est **essentiel pour inclure la commande dans un script**.
- Spécifie l'image Docker à utiliser pour exécuter les jobs.
- Définit l'exécuteur, c'est à dire l'environnement d'exécution utilisé par le runner.

Pour valider le succès de l'enregistrement, consultez la liste des runners de votre projet sur le serveur GitLab via le menu *Settings* > *CI/CD*.

Premier script

Hello World

Maintenant que le runner est configuré, créez **à la racine de votre projet** le fichier `.gitlab-ci.yml` suivant :

`.gitlab-ci.yml`

```
1 my_first_job:
2   script:
3     - echo "Hello World!"
```

Ce fichier définit un pipeline GitLab CI/CD avec un seul job nommé **my_first_job**.

Élément	Description
<code>my_first_job:</code>	Nom du job, qui apparaîtra dans l'interface GitLab CI/CD.
<code>script:</code>	Indique la ou les commandes à exécuter dans le job.

Élément	Description
<pre>- echo "Hello World!"</pre>	Commande exécutée dans l'environnement du runner. Elle affiche simplement <code>Hello World!</code> dans la console.

Déposez ce fichier sur le dépôt, via les étapes `git add .`, `git commit -m "Premier pipeline"` et `git push`.

Lorsqu'un commit est poussé dans le dépôt, GitLab CI/CD détecte le fichier `.gitlab-ci.yml`. Il crée alors un pipeline et exécute le job `my_first_job`. Le runner associé exécute le script qui affiche le message dans les logs. Le job se termine avec le statut *Success*, sauf en cas d'erreur.

Pour vérifier l'exécution du job, consultez sur la page de votre projet sur le serveur GitLab, le menu **Build > Jobs**. Cliquez ensuite sur le nom du job, ressemblant à `#7562: my_first_job`, afin de lire les logs de l'exécution. Cherchez dans ces logs le résultat de l'instruction `echo "Hello World!"`.

Variables du projet

Vous pouvez déclarer une variable dans le fichier `.gitlab-ci.yml` via le mot clé `variables` :

`.gitlab-ci.yml`

```
1 variables:
2   FIRST_VARIABLE: "Test"
3
4 deploy_job:
5   script:
6     - echo "Hello World!"
7     - echo "Contenu de la variable FIRST_VARIABLE $FIRST_VARIABLE"
```

Vous pouvez également définir une variable via l'interface GitLab. Dans le menu **Settings > CI/CD/ Variables**, il faut cliquer sur *Add Variable* et remplir les champs :

- Type : Variable d'environnement ou fichier.
- Environments : Appliquer à tous les environnements ou un spécifique. Les environnements peuvent être définis via le mot clé `environment` dans le script.
- Options :
 - Protected : Utilisable uniquement sur **les branches protégées**.

- Masked : Cachée dans les logs CI/CD.
- Key : nom de la variable, par exemple MY_VARIABLE
- Value : Valeur de la variable.

Lorsqu'une variable est de type *Fichier*, sa valeur est stockée dans un fichier temporaire sur le runner pendant l'exécution du job. Cette option est utile pour stocker des certificats ou des clés privées

Le champ *Masked* permet de cacher la valeur de la variable lorsqu'elle apparaît dans les logs des jobs ou dans l'interface. **Cette option est particulièrement utile pour protéger des informations sensibles** (comme des mots de passe, des clés API ou des tokens).

EXERCICE B : CRÉER UNE VARIABLE

Créez la variable MY_VARIABLE avec la valeur 42 via l'interface GitLab. Vérifiez la création de la variable via une mise à jour du fichier *gitlab-ci.yml*.

.gitlab-ci.yml

```
1 my_first_job:
2   script:
3     - echo "Hello World!"
4     - echo "La valeur de MY_VARIABLE est '$MY_VARIABLE'"
```

MISE À JOUR DU PIPELINE

Pour mettre à jour le fichier *gitlab-ci.yml* directement via l'interface GitLab, ouvrez le menu **Build > Pipeline editor** de votre projet. Une fois vos modifications rédigées, appuyez sur le bouton **Commit changes**.

Condition

Pour ajouter une condition sur une variable vous pouvez utiliser le bloc conditionnel de votre shell. Par exemple, le script peut ressembler à :

.gitlab-ci.yml

```
1 my_first_job:
2   script:
```

```
3 - echo "Hello World!"
4 - |
5     if [ "$MY_VARIABLE" -eq 42 ]; then
6         echo "La variable MY_VARIABLE vaut 42 comme attendu"
7     else
8         echo "Attention la variable MY_VARIABLE n'a pas la valeur prévue
$MY_VARIABLE"
9     fi
```



BLOCK STYLE INDICATOR

En YAML le *Block Style Indicator* est un mécanisme qui permet de spécifier un bloc de texte multiligne de manière plus lisible et structurée. L'indicateur `|` est utilisé pour indiquer que vous avez un texte multiligne. Les retours à la ligne et les indentations seront préservés dans le fichier YAML.

Variables prédéfinies

Plusieurs variables sont prédéfinies pour le runner. On trouve par exemple `CI_COMMIT_AUTHOR` contenant l'auteur du commit ou encore `CI_COMMIT_BRANCH` contenant la branche sur laquelle le commit a été effectué. Vous pouvez consulter la [liste des variables prédéfinies](#).



EXERCICE C : UTILISER DES VARIABLES PRÉDÉFINIES

Modifiez votre pipeline pour afficher les informations suivantes :

- Url du Projet
- Auteur du commit
- Branche de commit

Vérifiez le résultat de l'exécution de votre pipeline.

Execution failed

Dans cette section vous allez créer un pipeline qui va se terminer en échec afin d'expérimenter comment GitLab vous informe d'une erreur.



EXERCICE D : DÉTERMINER LE STATUT D'UN JOB

Modifiez votre pipeline en demandant au job d'afficher la version de maven utilisable dans le runner.

`.gitlab-ci.yml`

```
1 my_first_job:
2   script:
3     - mvn --version
```

Maven n'étant pas installé dans l'image de votre runner (comment le sait-on ? Quelle est cette image ?), vérifiez que le statut de votre job est **failed** via le menu **Build > Jobs**

Cherchez dans les logs la cause de l'erreur.

Before et after script

Si vous souhaitez exécuter une action avant ou après l'exécution des instructions contenues dans la partie script d'un job, vous pouvez ajouter les sections `before_script` et `after_script` comme dans l'exemple suivant :

`.gitlab-ci.yml`

```
1 my_first_job:
2   before_script:
3     - echo "Installation des dépendances..."
4     - apk update
5     - apk add curl
6     - echo "Dépendances installées. Début du job..."
7   script:
8     - echo "Exécution du job"
9   after_script:
10    - echo "Nettoyage des fichiers temporaires (si nécessaire)..."
11    - echo "Fin du job."
```

EXERCICE D : INSTALLER MAVEN

Afin de corriger le pipeline précédent, il faut installer maven dans votre runner. Modifiez ce pipeline pour installer maven via la directive `before_script`.

Default et image

Dans un fichier `.gitlab-ci.yml`, la directive `image:` est utilisée pour spécifier l'image Docker que GitLab CI/CD doit utiliser pour **exécuter le job**. Cette image définit l'environnement dans lequel les commandes du job seront exécutées (à la place de l'image par défaut définie lors de l'enregistrement du *runner*). Cela permet de personnaliser l'environnement d'exécution et de s'assurer que les outils ou dépendances nécessaires sont disponibles.

Par exemple vous pouvez utiliser une image spécifique pour un job nécessitant maven.

`.gitlab-ci.yml`

```
1 my_first_job:
2   image: maven:3.9.9-eclipse-temurin-23-alpine
3   script:
4     - mvn --version
```

Vous pouvez également définir une image par défaut pour le pipeline complet:

`.gitlab-ci.yml`

```
1 default:
2   image: maven:3.9.9-eclipse-temurin-23-alpine
3
4 my_first_job:
5   script:
6     - mvn --version
```

Définition d'un pipeline

Stages et dépendances

Dans un fichier `.gitlab-ci.yml`, la directive `stages` permet de définir les différentes étapes du pipeline. L'exécution de ces étapes est séquentielle, **tous les jobs d'un stage doivent être terminés avec succès** avant de passer au stage suivant.

Dans l'exemple ci-dessous, le pipeline est structuré en 3 étapes : `build`, `test`, et `deploy`. `build_job` s'exécute en premier, une fois terminé, `test_job` démarre et une fois les tests réussis, `deploy_job` démarre.

.gitlab-ci.yml

```
1  stages:
2    - build
3    - test
4    - deploy
5
6  build_job:
7    stage: build
8    script:
9      - echo "Compilation du projet..."
10
11 test_job:
12   stage: test
13   script:
14     - echo "Exécution des tests..."
15
16 deploy_job:
17   stage: deploy
18   script:
19     - echo "Déploiement en production..."
```

Plusieurs jobs peuvent être associés à un stage. Par exemple dans le cadre d'une application divisée en une partie frontend et backend, on peut imaginer le pipeline suivant :

.gitlab-ci.yml

```
1  stages:
2    - build
3    - deploy
4
5  # Deux jobs dans le stage "build"
6  compile_frontend:
7    stage: build
8    script:
9      - echo "Compilation du frontend..."
10
11 compile_backend:
12   stage: build
13   script:
14     - echo "Compilation du backend..."
15
16 # Un seul job dans le stage "deploy"
17 deploy_app:
```

```
18  stage: deploy
19  script:
20    - echo "Déploiement de l'application..."
```

💡 OPTIMISATION

Tous les jobs appartenant au même stage s'exécutent en parallèle si plusieurs runners sont disponibles.

❗ EXERCICE F : STRUCTURER UN PIPELINE

Écrivez un pipeline pour l'application *demo-no-db* qui.

- Définit deux stages : *test* et *build*.
- Crée deux jobs distincts (*job-test* et *job-build*)
- S'exécute dans un environnement Maven basé sur l'image `maven:3.9.9-eclipse-temurin-23-alpine`.
- Utilise `mvn test` pour exécuter les tests dans *job-test*.
- Utilise `mvn package -DskipTests` pour compiler le projet sans exécuter les tests dans *job-build*.

Échange de fichiers entre jobs

Dans GitLab CI/CD, un **artifact** est un fichier ou un ensemble de fichiers générés lors de l'exécution d'un job dans un pipeline. Ces fichiers peuvent être sauvegardés et partagés **entre différents jobs du pipeline**.

Par exemple si vous souhaitez que le résultat de la compilation par maven soit conservé pour le job de test, il suffit de modifier le job de compilation comme ci-dessous :

`.gitlab-ci.yml`

```
1  build:
2    stage: build
3    image: maven:3.9.9-eclipse-temurin-23-alpine
4    script:
5      - echo "Compilation du projet..."
6      - mvn compile
7    artifacts:
8      paths:
```

```
9      - target/ # Conserver Le dossier target
10     expire_in: 1h # L'artifact est supprimé après 1 heure
```

i EXERCICE : UTILISATION D'ARTIFACTS AVEC DEMO-NO-DB

Reprenez le pipeline pour l'application *demo-no-db*

- Définissez trois *stages* : *compile*, *test* et *build*.
- Utilisez les artifacts pour récupérer ce qui est utile pour les étapes suivantes.
- Comment récupérer le `.jar` produit à la dernière étape ?
- Récupérez-le et vérifiez que votre application se lance bien.

Ajout d'une cache

La directive `cache` dans un fichier `.gitlab-ci.yml` permet de mémoriser certains fichiers ou répertoires sur le runner entre les différentes exécutions d'un pipeline. Cela évite de réinstaller ou reconstruire des dépendances à chaque job, réduisant ainsi le temps d'exécution du pipeline.

Par exemple Maven télécharge les dépendances d'un projet depuis un repository distant et les stocke localement dans le répertoire `~/.m2/repository`. Sans cache, chaque job Maven dans un pipeline re-téléchargera toutes les dépendances, ce qui est lent et consomme de la bande passante. Le pipeline ci-dessous montre comment utiliser ce cache dans le cadre d'un job maven.

`.gitlab-ci.yml`

```
1 my_first_job:
2   image: maven:3.9.9-eclipse-temurin-23-alpine
3   cache:
4     paths:
5       - .m2/repository/
6   script:
7     - mvn test -Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository
```

i EXERCICE G : OPTIMISER LES INSTALLATIONS

Créez un pipeline GitLab CI/CD qui exécute le script Python suivant :

```
import requests

response = requests.get("https://git.esi-
```

```
bru.be/api/v4/broadcast_messages")
if response.status_code == 200:
    print("Requête réussie :", response.json())
else:
    print("Erreur :", response.status_code)
```

Le script nécessite un environnement *Python 3.8* et l'installation de la bibliothèque *requests* via la commande `pip install requests`.

1. Configurez le pipeline pour installer *requests* dans un environnement Docker avec l'image *python:3.8*.
2. Utilisez la directive `cache` pour stocker les dépendances installées dans le répertoire `~/.cache/pip`.
3. Vérifiez que le cache fonctionne en exécutant plusieurs fois le pipeline, en vous assurant que les dépendances ne sont pas téléchargées à chaque exécution.

Exécution conditionnée

Contrôler un job avec when

Placée dans un job la directive `when` permet de contrôler quand ce job doit être exécuté.

Dans l'exemple ci-dessous, l'utilisation de `when: on_failure` signifie que le job ne sera exécuté que si l'un des jobs précédents échoue. Cela peut être particulièrement utile pour des actions telles que l'envoi de notifications en cas d'échec ou des processus de récupération, comme le rollback d'une base de données ou l'installation d'une version précédente de l'application si l'installation de la nouvelle version échoue.

`.gitlab-ci.yml`

```
1 stages:
2   - build
3   - test
4   - notify
5
6 build-job:
7   stage: build
8   script:
9     - echo "Compilation du projet..."
10    - exit 1 # Simule un échec
11
```

```
12 test-job:
13   stage: test
14   script:
15     - echo "Lancement des tests..."
16
17 notify-job:
18   stage: notify
19   script:
20     - echo "Envoi de la notification d'échec..."
21   when: on_failure # Ce job s'exécute si un job précédent échoue
```

EXERCICE H : CRÉER UN JOB DE RÉCUPÉRATION

Exécutez le pipeline ci-dessus et vérifiez que le job `notify-job` est exécuté et que le job `test-job` ne démarre jamais via le menu *Build > Pipelines* de votre projet sur le serveur GitLab.

Modifiez le job `build-job` et ajoutez la directive `allow_failure: true`. Quelle modification du comportement du pipeline observez-vous ?

Consultez les différentes valeurs possibles pour la directive `when` dans la [documentation](#).

VALEUR PAR DÉFAUT

Par défaut chaque job est associé aux directives `when: on_success` et `allow_failure: false`. Ce qui explique que chaque job s'exécute si aucun job précédent n'est tombé en erreur.

Contrôler un pipeline avec workflow

La directive `workflow` dans un fichier `.gitlab-ci.yml` permet de définir les conditions sous lesquelles un pipeline GitLab doit être exécuté. Elle sert à contrôler l'exécution du pipeline dans son ensemble, en fonction de conditions telles que des branches spécifiques, des événements de merge request, ou d'autres critères.

ONLY/EXCEPT

Les règles de contrôle d'exécution peuvent être définies via les directives *only* et *except*. Mais ces directives sont signalées comme deprecated et doivent être évitées.

Dans l'exemple ci-dessous, le pipeline se déclenche seulement si le commit est sur la branche `main`. Cela permet de restreindre l'exécution des pipelines à des cas spécifiques, comme un déploiement sur la branche principale.

`.gitlab-ci.yml`

```
1 workflow:
2   rules:
3     - if: '$CI_COMMIT_BRANCH == "main"'
4
5   stages:
6     - deploy
7
8   deploy-job:
9     stage: deploy
10    script:
11      - echo "Déploiement de la version"
```

`rules:` permet de spécifier des règles basées sur des variables d'environnement (comme `$CI_COMMIT_BRANCH`, `$CI_COMMIT_TAG`,...). Chaque règle est précisée via l'utilisation de `- if:`.

i EXERCICE H : DÉPLOYER UNE APPLICATION SOUS CONDITION

Un tag dans Git est une référence ou un marqueur attaché à un commit spécifique dans l'historique d'un projet. Contrairement à une branche, un tag est généralement utilisé pour marquer des points importants dans l'historique, comme des versions ou des releases. Les commandes suivantes permettent de créer un tag associé à la version 1.0 du projet.

```
git tag v1.0
git push origin v1.0
```

Créez un pipeline qui ne s'exécute que lorsqu'un tag correspondant au format de version v1.0, v2.0, v3.0, etc., est ajouté au dépôt. Une expression régulière du type `^v\d+\.\d+$` peut être utilisée pour reconnaître le format.

Analyse avec SonarQube

Une étape importante dans l'exécution d'un pipeline est l'analyse du code déposé. Le GitLab Runner, chargé d'exécuter les demandes du script, crée **un nouveau conteneur** basé sur l'image spécifiée par la directive `image:`.

Pour permettre la communication entre le serveur SonarQube, créé lors du TD précédent, et ce conteneur temporaire, ils doivent être connectés au même réseau. Vous aviez appelé ce réseau `sonar-network`.

La solution la plus simple, sans modifier manuellement le fichier de configuration situé dans le dossier `gitlab-runner/config`, consiste à :

- Retirer le GitLab Runner de la liste des runners de votre projet via l'interface web de git.esi-bru.be.
- Arrêter le conteneur GitLab Runner avec la commande : `docker stop gitlab-runner`
- Supprimer le conteneur GitLab Runner avec : `docker rm gitlab-runner`
- Créer un nouveau conteneur GitLab Runner (la commande n'a pas changé):

```
docker run -d --name gitlab-runner --restart always \
  -v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-runner \
  -v //var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest
```

RAPPEL

Le `chemin_absolu_vers_la_configuration` est `/srv` sur Linux et un dossier créé par vous sur Windows.

- Enregistrer le runner en précisant le réseau auquel doivent être connectés tous les conteneurs utilisés, à l'aide de la commande

```
docker run --rm \
  -v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-runner \
  gitlab/gitlab-runner register \
    --non-interactive \
    --url "https://git.esi-bru.be" \
    --token "$RUNNER_TOKEN" \
    --executor "docker" \
    --docker-image alpine:latest \
    --description "docker-runner" \
    --docker-network-mode "sonar-network"
```

EXERCICE I : ANALYSER LA QUALITÉ DU CODE

Créez un pipeline pour l'application *demo-no-db* qui lance les tests de l'application et analyse le code via SonarQube.

Pour permettre au runner d'enregistrer les résultats de l'analyse de code à SonarQube, créez **la variable secrète** `SONAR_TOKEN` sur le serveur git contenant le token d'authentification au serveur SonarQube.

Le stage de compilation a utiliser est :

.gitlab-ci.yml

```
1 build:
2   stage: build
3   image: maven:3.9.9-eclipse-temurin-23-alpine
4   script:
5     - echo "Compilation du projet..."
6     - mvn compile
7   artifacts:
8     paths:
9       - target/ # Sauvegarde les fichiers compilés
10    expire_in: 1h
```

Le stage de test peut se résumer à :

.gitlab-ci.yml

```
1 test:
2   stage: test
3   image: maven:3.9.9-eclipse-temurin-23-alpine
4   script:
5     - echo "Exécution des tests avec JaCoCo..."
6     - mvn test jacoco:report
7   artifacts:
8     paths:
9       - target/site/jacoco/ # Sauvegarde Le rapport JaCoCo
10    expire_in: 1h
```

Le stage d'analyse peut s'écrire comme :

.gitlab-ci.yml

```
1 sonarqube:
```

```
2  stage: sonarqube
3  image:
4      name: sonarsource/sonar-scanner-cli:latest
5  script:
6      - echo "Exécution de l'analyse via SonarQube"
7      - sonar-scanner -Dsonar.host.url=http://sonarqube:9000 -
      Dsonar.login=$SONAR_TOKEN
```

Vérifiez le bon fonctionnement de votre pipeline en consultant le résultat de l'analyse via <http://sonarqube:9000>.

Déploiement sur Alwaysdata

Lors des précédents TDs sur l'automatisation du déploiement sur AlwaysData, vous avez configuré une connexion SSH entre votre machine hôte et AlwaysData en générant une clé SSH. La clé privée associée à cette connexion est stockée sur votre machine dans le fichier `~/.ssh/id_rsa`.

Pour automatiser le déploiement de l'application `demo-no-db` via un pipeline, le runner doit utiliser la commande `scp` pour transférer le fichier `.jar` généré après la compilation.

Afin d'établir une connexion sécurisée entre le runner et AlwaysData, vous devez ajouter votre clé privée SSH en tant que variable masquée dans GitLab CI/CD sous le nom `SSH_PRIVATE_KEY`. Le pipeline pourra ensuite récupérer cette clé et l'utiliser grâce aux commandes suivantes :

```
echo "$SSH_PRIVATE_KEY" > ~/.ssh/id_rsa
chmod 600 ~/.ssh/id_rsa
```

Ces instructions permettront au runner de s'authentifier et d'effectuer le transfert du fichier sans intervention manuelle.

EXERCICE J : DÉPLOYER SUR ALWAYSDATA

Créez un pipeline pour l'application *demo-no-db* qui :

- Compile l'application et génère le fichier **.jar**.
- Déploie automatiquement l'application sur AlwaysData.

Si le fichier **.jar** n'est pas disponible après la compilation, consultez [la documentation sur le mot-clé artifacts](#).

Bonus : La documentation d'AlwaysData indique qu'il est possible de redémarrer le serveur via une API. Essayez d'intégrer cette fonctionnalité à votre pipeline pour déclencher automatiquement le redémarrage du serveur après le déploiement.

Docker dans un pipeline

Lorsqu'on veut exécuter docker à l'intérieur d'un conteneur docker, deux solutions existent :

- Docker in Docker (DinD) désigne l'exécution du démon Docker à l'intérieur d'un conteneur Docker. Ce processus lance des conteneurs "enfants". Cette technique requiert de gérer certains privilèges du conteneur et peut se faire via l'image docker "docker".
- Donner accès au démon Docker hôte afin que le conteneur puisse lancer des conteneurs "frères et sœurs". Cette technique requiert d'avoir un client docker dans le conteneur, et de donner au conteneur un moyen de communiquer avec le démon Docker de l'hôte.

❗ DÉJÀ VU!

Nous avons déjà utilisé la seconde technique pour que le conteneur gitlab-runner puisse lancer les images de nos pipelines : c'est le rôle de l'option `-v /var/run/docker.sock:/var/run/docker.sock` utilisée jusqu'à présent. Dans les paragraphes à venir, nous allons mettre en place un équivalent à cette option pour lancer nos pipelines.

Variante "accès au démon hôte"

Voici un exemple de pipeline simple utilisant docker :

`.gitlab-ci.yml`

```
1 build:
2   image: alpine:latest
3   script:
4     - apk add docker-cli
5     - printf '%s\n' "FROM alpine" "ENTRYPOINT echo coucou" >Dockerfile
6     - docker build -t mon-test-depuis-un-conteneur .
```

Testez-le maintenant : il doit échouer avec un message indiquant que le démon docker n'est pas accessible.

❗ SOUS WINDOWS - DOCKER.SOCK

Sous Windows, Docker fonctionne via Docker Desktop, qui crée une machine virtuelle Linux pour exécuter les conteneurs. C'est dans cette VM que se trouve le fichier `/var/run/docker.sock` qui permet de communiquer avec le démon docker.

❗ SOUS WINDOWS - GIT BASH ET `//`

Une autre complication est liée à Git Bash : pour nous aider, ce dernier ajuste automatiquement certains chemins qui commencent par `/`. C'est la raison pour laquelle vous voyez `//` apparaître dans de nombreuses commandes de ce labo : cette astuce désactive l'ajustement automatique de Git Bash.

Plus d'information sur [la documentation de MSYS2](#). (L'astuce utilisant `//` n'est pas mentionnée mais son équivalent moderne avec la variable `MSYS2_ENV_CONV_EXCL` l'est.)

Pour permettre à votre conteneur exécutant le job (appelé « Executor ») d'utiliser lui-même Docker, [la documentation mentionne](#) qu'il faut enregistrer votre runner avec le volume `/var/run/docker.sock:/var/run/docker.sock`. La solution la plus simple, sans modifier manuellement le fichier de configuration situé dans le dossier `gitlab-runner/config`, consiste à :

- Arrêter le conteneur GitLab Runner avec la commande : `docker stop gitlab-runner` (rappel: en le créant, vous aviez utilisé l'option `--rm` donc le conteneur sera également supprimé en s'arrêtant.)
- Supprimer le fichier de configuration `//chemin_absolu_vers_la_configuration/gitlab-runner/config.toml`
- Créer un nouveau conteneur GitLab Runner (la commande n'a pas changé):

```
docker run -d --name gitlab-runner --restart always \
  -v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-runner \
  -v //var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest
```

- Enregistrer le runner en précisant le volume vers le socket Docker, à l'aide de la commande :

```
docker run --rm \
  -v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-
```

```
runner \  
  gitlab/gitlab-runner register \  
    --non-interactive \  
    --url "https://git.esi-bru.be" \  
    --token "$RUNNER_TOKEN" \  
    --executor "docker" \  
    --docker-image alpine:latest \  
    --description "docker-runner" \  
    --docker-volumes //var/run/docker.sock:/var/run/docker.sock
```

Désormais le pipeline ci-dessus devrait passer (il faudra le relancer à la main ou refaire un commit). Par ailleurs, puisque le démon docker utilisé pour build est **le même** que celui de l'hôte Windows, vous **devriez voir** l'image avec la commande `docker image ls`.

Variante Docker in Docker

Voici un exemple de pipeline simple utilisant DinD :

`.gitlab-ci.yml`

```
1 build:  
2   image: docker:latest  
3   services:  
4     - docker:dind  
5   script:  
6     - printf '%s\n' "FROM alpine" "ENTRYPOINT echo coucou" >Dockerfile  
7     - docker build -t mon-test-depuis-dind .
```

Testez-le maintenant : il doit échouer avec un message indiquant que le service dind ne peut pas s'exécuter et que le démon docker n'est pas accessible.

Pour permettre à votre conteneur exécutant le job (appelé « Executor ») d'utiliser DinD, [la documentation mentionne](#) qu'il faut enregistrer votre runner pour qu'il crée des conteneurs privilégiés. La solution la plus simple, sans modifier manuellement le fichier de configuration situé dans le dossier `gitlab-runner/config`, consiste à :

- Arrêter le conteneur GitLab Runner avec la commande : `docker stop gitlab-runner` (rappel: en le créant, vous aviez utilisé l'option `--rm` donc le conteneur sera également supprimé en s'arrêtant.)
- Supprimer le fichier de configuration `//chemin_absolu_vers_la_configuration/gitlab-runner/config.toml`

- Créer un nouveau conteneur GitLab Runner (la commande n'a pas changé):

```
docker run -d --name gitlab-runner --restart always \
-v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-
runner \
-v //var/run/docker.sock:/var/run/docker.sock \
gitlab/gitlab-runner:latest
```

- Enregistrer le runner avec les options indiquant de créer des conteneurs privilégiés et donnant accès aux certificats TLS, à l'aide de la commande :

```
docker run --rm \
-v //chemin_absolu_vers_la_configuration/gitlab-runner/config:/etc/gitlab-
runner \
gitlab/gitlab-runner register \
--non-interactive \
--url "https://git.esi-bru.be" \
--token "$RUNNER_TOKEN" \
--executor "docker" \
--docker-image alpine:latest \
--description "docker-runner" \
--docker-privileged \
--docker-volumes //certs/client
```

Désormais le pipeline ci-dessus devrait passer (il faudra le relancer à la main ou refaire un commit). Par ailleurs, puisque le démon docker utilisé pour build est **différent** de celui de l'hôte Windows, vous ne verrez **pas** l'image avec la commande `docker image ls`.

Conclusion

L'utilisation de docker dans des conteneurs sera implémentée lors des scénarios de synthèse qui récapitulent l'ensemble des étapes vues dans les TDs.